MIT 8103 Advanced Database Systems

Inventory and Supply Chain Management System · CHUKWUMA UGOBUEZE · Student ID 30164952

Case study

My case study is Inventory and Supply Chain Management System. I designed the database for inventory staff who review stock levels and record events, procurement staff who manage suppliers and purchase orders and warehouse managers who review stock by location, with separate tables named warehouse, item, supplier, inventory_balance, purchase_order and stock_event. It supports these operations: create a purchase order for one item from a supplier to a warehouse; record receipt, transfer, or adjustment events and update current stock; find balances below their reorder points and summarize receipts by warehouse and item; benchmark the same synthetic event-date filter before and after creating a B-tree index, using 20000 synthetic event rows and recording the query plan and elapsed time for each run.

Portfolio 1: Database Design and Modelling

I organised the relational design into 6 tables. Primary keys identify records, foreign keys preserve the links, and checks enforce the stated business rules. On-hand quantity and reorder point must each be zero or greater; stock updates lock and recheck the balance row so concurrent changes cannot produce a negative on-hand quantity. The normalisation examples show the sample data at 1NF, 2NF and 3NF. All 24 deliberately invalid inserts were rejected without changing the valid sample rows.

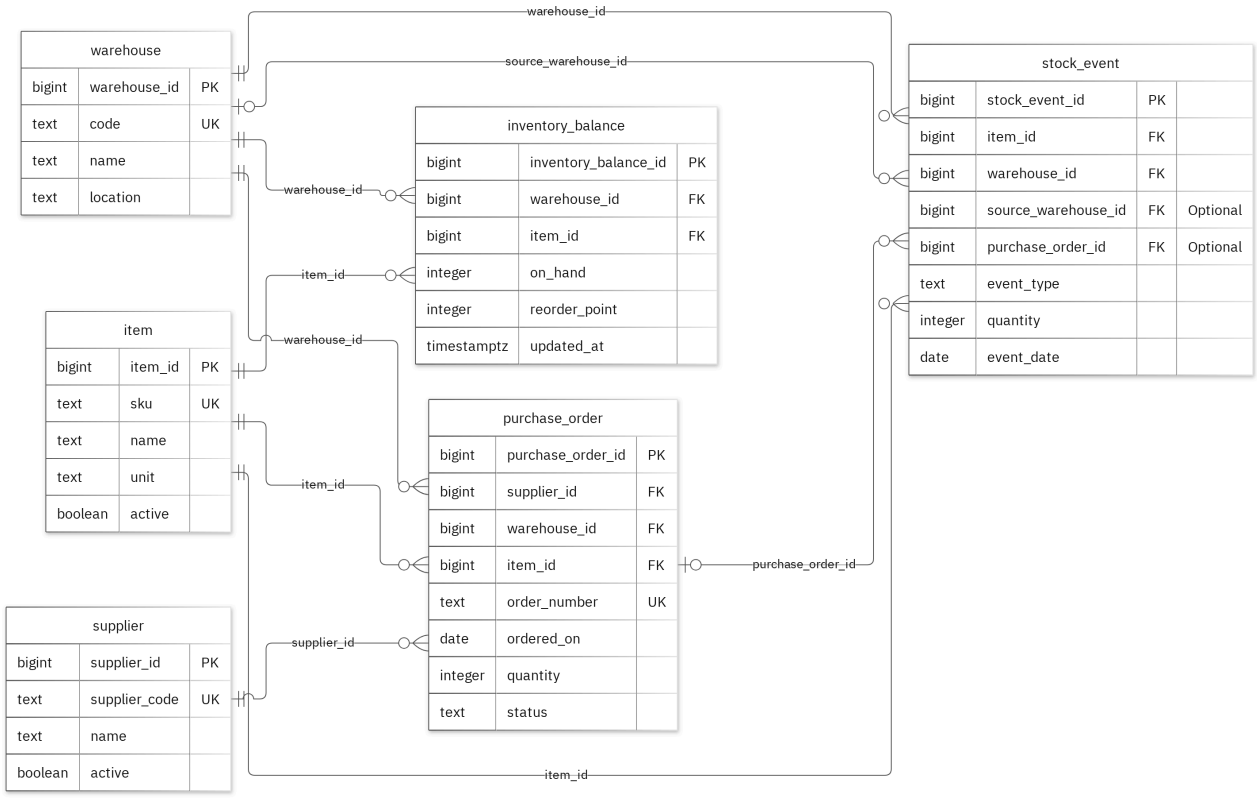


Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I ran three SQL queries against the sample records. The join returned 1 rows and included PO-2026-001 with SUP-A. For the index comparison, I used 20,000 separate performance rows. The median lookup time changed from 0.939 ms to 0.203 ms. Both phases returned the same records. The measured plan used the new index. These timings apply to this lookup and environment.

Portfolio 3: Transactions and Concurrency

I used a temporary counter on inventory_balance to test COMMIT, ROLLBACK and READ COMMITTED isolation. The uncommitted update remained hidden from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE, Session B waited for Session A and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used inventory_events for the case's event documents. MongoDB inserted 4 sample events and returned the expected results for 4 queries. Validation rejected an invalid date type and a duplicate event ID. Embedded snapshots keep related details with an event, while the details field can differ by event type. These are fictional events; PostgreSQL continues to enforce its relational rules.

Repository and evidence

https://github.com/Chusoch/UGOBUEZE_CHUKWUMA_30164952

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed three MongoDB replica members across separate failure zones. The discussion covers primary elections, majority acknowledgement, secondary lag and sharding trade-offs for inventory_events. Replication supports availability, while backups address recovery. PostgreSQL needs its own availability plan. This is the permitted discussion option; no live replica set or failover test was deployed.

Individual reflection

I kept warehouse, item, supplier, inventory_balance, purchase_order, stock_event in separate tables so their details could be maintained without repeating them in every related record. On-hand quantity and reorder point must each be zero or greater; stock updates lock and recheck the balance row so concurrent changes cannot produce a negative on-hand quantity. The constraint tests check this alongside the required values and record links. One issue illustrated by the two-session exercise is that both sessions can read the same value before either update is committed. Locking the row before reading preserves both increments. The index comparison also shows why I need to check the execution plan as well as the final answer: the same result can require different amounts of database work.

AI-use declaration

I used GitHub Copilot and the portfolio generator to help prepare the database model, scripts and explanations. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images. I remain responsible for checking and explaining the submitted work.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks
- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams